# A gm/Id Methodology Design Tool for Analog CMOS Circuit Synthesis: Application to the Folded-Cascode Class-AB Amplifier

Phil Alcorn
*Department of Electrical Engineering*
*Tarleton State University*
Stephenville, TX
palcorn2000@gmail.com

***Abstract*—This paper presents a Python-based simulation pipeline that automates the gm/Id transistor sizing methodology for CMOS analog circuits. The tool characterizes NMOS and PMOS transistors across channel length, gate-to-source voltage, drain-to-source voltage, and bulk-to-source voltage using LTspice operating-point sweeps, stores the results in a four-dimensional look-up table (LUT), and provides a query interface for transistor sizing. The design flow is demonstrated by applying the extracted LUTs to the sizing of a folded-cascode class-AB operational amplifier, walking through each transistor group from specification to final width.**

***Index Terms*—gm/Id methodology, CMOS amplifier design, LTspice automation, look-up table, folded-cascode amplifier, class-AB output stage, transistor sizing**

## I. Introduction

Traditional analog design involves using the square law to derive sizing equations. The square-law predicts that in saturation

$$I_D = \frac{1}{2}k'\frac{W}{L}(V_{GS} - V_{th})^2, \tag{1}$$

and sizing a transistor means solving (1) for $W/L$ given a target $I_D$ and an assumed overdrive $V_{on} = V_{GS} - V_{th}$. In practice, modern SPICE models include effects that make the real device behave differently from this simple equation, which is more accurate to real performance. As a result of this, transistors sized by hand from (1) will not match their simulated operating point. There is therefore a necessary iteration state where designers work back and forth between hand calculations and simulation results until there is a reasonable amount of convergence between the two. This is very time consuming.

The gm/Id methodology [1] bypasses this iteration by replacing the square-law with data taken directly from the SPICE model. Rather than computing $W$ from an equation, the designer looks up how much current a given width carries at the desired bias condition and scales accordingly. Because the data already include all of the model's non-ideal behavior, no iteration is needed. Murmann's ISSCC lecture [2] provides an accessible introduction to the methodology.

This work implements the gm/Id workflow in Python on top of PyLTSpice [3], enabling automated parametric sweeps, structured data extraction, and an interactive query interface.

## II. The gm/Id Design Methodology

### A. Transconductance Efficiency

The transconductance $g_m$ of a MOSFET is defined as the change in drain current per change in gate voltage, $g_m = \partial I_D/\partial V_{GS}$. Dividing by the quiescent drain current gives the *transconductance efficiency*:

$$\frac{g_m}{I_D}\ \left[\mathrm{V}^{-1}\right]. \tag{2}$$

This ratio can answer several design questions simultanously, and can be used as the "knob" to find the balance between conflicting design parameters, such as when trying to maximize bandwidth while minimizing current requirements.

From the square-law, $g_m = 2I_D/V_{on}$, so

$$\left.\frac{g_m}{I_D}\right|_{\text{sq.law}} = \frac{2}{V_{on}}. \tag{3}$$

As $V_{GS}$ increases, and the device is driven harder into strong inversion, $V_{on}$ grows and $g_m/I_D$ falls. As $V_{GS}$ approaches $V_{th}$ the device enters weak inversion (subthreshold), where $I_D$ is exponential and $g_m/I_D$ reaches its maximum of roughly 25–$35\,\mathrm{V}^{-1}$ for a typical process.

In reality, $g_m/I_D$ is a smooth function of $V_{GS}$ that does not follow (3) exactly. The LUT captures this real curve directly from simulation data.

### B. What the gm/Id Value Tells You

Choosing a $g_m/I_D$ target is equivalent to choosing where on the weak-inversion to strong-inversion continuum the transistor will be biased:

- *Weak inversion* (high $g_m/I_D$, near the technology maximum): maximum power efficiency. The transistor draws

very little current, but it is slow and $V_{dsat}$ is large (poor output swing).

- *Moderate inversion* (mid-range $g_m/I_D$): good balance of efficiency and speed. A practical starting point for most signal-path transistors.
- *Strong inversion* (low $g_m/I_D$): lower efficiency, but the device is fast and $V_{dsat}$ is small, which is useful for output-stage transistors where output swing matters.

Once a $g_m/I_D$ target is chosen, the LUT gives the drain current per unit width ($I_D/W$) at that operating point. Knowing the required $I_D$ from the circuit specification, the width follows directly:

$$W = \frac{I_{D,\text{required}}}{(I_D/W)_{\text{LUT}}}. \tag{4}$$

No iteration is directly required, but it is possible to work to minimize the current consumption my lowering $gm/I_D$. This is due to the fact that current density $I_D/w$ scales inversly with $gm/I_D$, as shown in [relevant graph].

### C. Key Design Variables in the LUT

For each bias point the LUT stores the following quantities (per unit width, for $W_{\text{ref}} = 10\,\mu$m):

- $I_D$, $g_m$, $g_{ds}$ — drain current, transconductance, and output conductance from the SPICE operating-point solver.
- $g_m/I_D$ — transconductance efficiency (computed).
- $I_D/W$ — current density (computed from $I_D$ and $W_{\text{ref}}$).
- $g_m/g_{ds}$ — intrinsic gain; useful for estimating $A_v$ of a single-transistor stage.
- $f_T = g_m/(2\pi C_{gg})$ — transition frequency; a measure of how fast the transistor is.
- $V_{th}$, $V_{dsat}$ — threshold and saturation voltage.

## III. Software Architecture

The pipeline is implemented in Python and uses PyLTSpice [3] to drive LTspice. Four modules handle the workflow.

**`spice.py`** wraps the PyLTSpice simulation runner. It converts the `.asc` schematic to a netlist, updates `.param` values for each sweep point, and runs up to 20 LTspice processes in parallel.

**`callbacks.py`** runs after each simulation finishes. It reads the `.log` operating-point data and the `.raw` waveform file, and bundles everything into a `SimResult` object keyed by device name.

**`parser.py`** converts a list of `SimResult` objects into structured `ParsedSimResult` objects containing the traces, measurements, and device parameters the designer requested.

**`characterization.py`** coordinates the full characterization sweep, assembles results into four-dimensional NumPy arrays indexed by $(L, V_{GS}, V_{DS}, V_{SB})$, and saves them to `.npz` files. **`query_lut.py`** loads a saved LUT and answers sizing queries via command-line arguments or an interactive terminal prompt.

## IV. Device Characterization Sweep

### A. Testbench Topology

The characterization schematic (`nmos_char.asc`) contains one NMOS (M1) and one PMOS (M2) transistor. Both are characterized in a single operating-point simulation by mirroring the NMOS bias about the supply rail for the PMOS:

$$V_{g,p} = V_{DD} - V_{gs}, \tag{5}$$
$$V_{d,p} = V_{DD} - V_{ds}, \tag{6}$$
$$V_{b,p} = V_{DD} - V_b. \tag{7}$$

This means that sweeping $(V_{gs}, V_{ds})$ for the NMOS simultaneously sweeps the intrinsic $(V_{SG}, V_{SD})$ of the PMOS, so one Python loop produces LUTs for both device types.

### B. Sweep Parameters

- $L \in \{0.30, 0.70, 1.00, 2.00\}\,\mu$m
- $V_{GS}$ from 0 to 5 V in 50 mV steps (101 points)
- $V_{DS}$ from 0 to 5 V in 50 mV steps (101 points)
- $V_{SB} \in \{0, 0.45, 0.90\}$ V
- Reference width $W_{\text{ref}} = 10\,\mu$m

This produces $4 \times 101 \times 101 \times 3 = 122\,412$ operating-point simulations run in parallel across 20 LTspice worker processes.

### C. Extracted Fields

TABLE I
Operating-Point Parameters Extracted Per Device

| Symbol | SPICE name | Description |
|---|---|---|
| $I_D$ | `Id` | Drain current |
| $g_m$ | `Gm` | Transconductance |
| $g_{ds}$ | `Gds` | Output conductance ($1/r_ds$) |
| $r_{ds}$ *(calculated)* | `Gds` | Output resistance |
| $g_{mb}$ | `Gmb` | Body transconductance |
| $C_{gg}$ | `dQgdVgb` + overlap | Total gate capacitance |
| $V_{th}$ | `Vth` | Threshold voltage |
| $V_{dsat}$ | `Vdsat` | Minimum $V_{DS}$ for saturation |

## V. Look-Up Table and Sizing Queries

### A. LUT Structure

After all simulations complete, the results are assembled into four-dimensional arrays of shape $(n_L, n_{Vgs}, n_{Vds}, n_{Vsb})$. The following derived quantities are appended:

$$\frac{g_m}{I_D}, \quad \frac{I_D}{W}, \quad \frac{g_m}{g_{ds}}, \quad f_T = \frac{g_m}{2\pi C_{gg}}. \tag{8}$$

Both NMOS and PMOS LUTs are saved to compressed NumPy archives (`nmos_lut.npz` and `pmos_lut.npz`).

### B. Query Modes

`query_lut.py` supports two usage modes.

**Bias mode:** Provide $(L, V_{GS}, V_{DS})$ and get all operating-point quantities back. Use this to check what a transistor looks like at a specific bias point.

```
python query_lut.py --l 1.0 --vgs 0.8 --vds
    0.9
```

**Design mode:** Provide a $g_m/I_D$ target, $L$, and $V_{DS}$, and optionally a target $g_m$ or $I_D$. The tool finds the $V_{GS}$ that hits the efficiency target, reads the current density from the LUT, and returns the required width using (4).

```
python query_lut.py --l 1.0 --gmid 15 --vds
    0.9 \
  --gm 628e-6
```

The design mode output includes $V_{GS}$, $I_D$, $W$, $g_m/g_{ds}$, $V_{dsat}$, and $f_T$ — all the values needed to verify a design decision and move on to the next transistor.

## VI. Application: Folded-Cascode Class-AB Amplifier

The $g_m/I_D$ methodology was applied to the cascode and current-source devices and the class-AB output stage. Each group was sized by querying the LUT at the target bias point rather than by solving the square-law model.

### A. Cascode and Current-Source Devices

The DC bias current for each cascode branch is $I_D = 10\,\mu\text{A}$. The supply-headroom budget sets a target saturation voltage $V_{dsat} \approx 122\,\text{mV}$ and a target drain-to-source voltage $V_{DS} \approx 160\,\text{mV}$. Minimum channel length $L = 0.30\,\mu\text{m}$ was chosen to maximise current density.

**NMOS cascode.** `query_lut.py` was run at $L = 0.30\,\mu\text{m}$, $V_{DS} = 0.15\,\text{V}$, sweeping $V_{GS}$ to find the lowest-$V_{dsat}$ operating point still in saturation. The best result was:

- $V_{GS} = 800\,\text{mV}$
- $g_m/I_D = 6.742\,\text{V}^{-1}$
- $I_D/W = 4.657\,\mu\text{A}/\mu\text{m}$

Applying (4):

$$W = \frac{10\,\mu\text{A}}{4.657\,\mu\text{A}/\mu\text{m}} \approx 2.14\,\mu\text{m}. \tag{9}$$

**PMOS cascode.** An identical sweep on the PMOS data ($L = 0.30\,\mu\text{m}$, $V_{SD} = 0.15\,\text{V}$, varying $V_{SG}$) returned:

- $V_{SG} = 0.70\,\text{V}$
- $g_m/I_D = 7.36\,\text{V}^{-1}$
- $I_D/W = 0.885\,\mu\text{A}/\mu\text{m}$

$$W = \frac{10\,\mu\text{A}}{0.885\,\mu\text{A}/\mu\text{m}} \approx 11.30\,\mu\text{m}. \tag{10}$$

The lower current density of the PMOS device relative to the NMOS reflects its lower carrier mobility; the wider device compensates to deliver the same drain current at the same $V_{dsat}$ budget.

### B. Class-AB Output Stage

**Determining $I_{max}$.** The peak output current is set by the load specification. With a 1.2 V peak swing into $R_L = 100\,\Omega$ and quiescent contributions from the bias network:

$$I_{max} = \frac{V_{pk}}{R_L} + 600\,\mu\text{A} + I_a \approx 12\,\text{mA} + 2.6\,\text{mA} \approx 14.6\,\text{mA}. \tag{11}$$

**Minimum $V_{DS}$.** The minimum drain-to-source voltage available to each output transistor follows from the supply headroom:

$$V_{DS,min} = 1.65\,\text{V} - 1.0\,\text{V}\,(1 + 0.25) = 400\,\text{mV}. \tag{12}$$

Minimum channel length ($L = 0.30\,\mu\text{m}$) was used to minimise device area given the large required width.

**NMOS output device — inversion-level selection.** The large $I_{max}$ means that the choice of $g_m/I_D$ directly controls total device width. Three targets were evaluated at $L = 0.30\,\mu\text{m}$, $V_{DS} = 400\,\text{mV}$:

TABLE II
NMOS Output Device: $g_m/I_D$ Trade-off ($L = 0.30\,\mu\text{m}$, $V_{DS} = 400\,\text{mV}$)

| $g_m/I_D$ ($\text{V}^{-1}$) | $I_D/W$ ($\mu$A/$\mu$m) | $g_m/g_{ds}$ | Total $W$ ($\mu$m) |
|---|---|---|---|
| 10 | 1.121 | 22.6 | 13 469 |
| 7 | 18.34 | 13.0 | 785 |
| 5 | 28.25 | 7.4 | 529 |

$g_m/I_D = 10$ was rejected because the resulting total width of 13,469 $\mu$m is impractically large. Reducing to $g_m/I_D = 7$ brought the total width to 785 $\mu$m but limited intrinsic gain to $g_m/g_{ds} = 13$ V/V. Either $g_m/I_D$ ratio could be selected depending on the design priorities.

**PMOS output device.** The same $g_m/I_D = 5$ target was applied to the PMOS. The LUT returned $I_D/W = 7.47\,\mu\text{A}/\mu\text{m}$, giving

$$W_{total} = \frac{14.6\,\text{mA}}{7.47\,\mu\text{A}/\mu\text{m}} \approx 1{,}918\,\mu\text{m}. \tag{13}$$

Individual device finger widths were set so that $M_{p,out} = M_{n,out}$, ensuring a symmetric layout for the output pair.

## VII. Representative LUT Plots

Four plots generated by `view_lut.py` illustrate the NMOS design space. These plots can also be generated for the PMOS devices by specifying a path to the PMOS numpy data file. All curves are parametrized by channel length $L \in \{0.30, 0.70, 1.00, 2.00\}\,\mu\text{m}$. The primary purpose of these graphs is to give designers a quick visualization for where tradeoffs happen. For example, one can quickly compare the bandwith that gets sacrificed when trying to maximize gain, and can visually get a quick estimate for an starting $g_m/I_D$.

**Fig. 1 — $I_D/W$ vs. gm/Id:** This is the primary sizing chart used in (4). Given a $g_m/I_D$ target, read the current density $I_D/W$ from the vertical axis. Dividing the required drain current by this density gives the transistor width directly. Shorter channels carry more current per micron at the same

$g_m/I_D$ target, which means a narrower device but at the cost of lower intrinsic gain.

**Fig. 2 — $g_m/g_{ds}$ vs. gm/Id:** This plot shows the intrinsic gain of a single transistor as a function of inversion level. Use it during Step 7 to estimate the cascode output resistance. Longer channels provide substantially higher intrinsic gain; the $L = 2.0\,\mu$m curve peaks near 650 V/V at moderate inversion.

**Fig. 3 — $g_m r_o$ vs. $V_{DS}$:** Intrinsic gain also depends on $V_{DS}$. This plot (at fixed $V_{ov} = 200\,$mV) shows that gain peaks around $V_{DS} \approx 2\,$V and falls at low $V_{DS}$ (where the device is near the triode boundary) and at high $V_{DS}$ (where channel-length modulation increases $g_{ds}$). Use it to verify that the chosen $V_{DS}$ for each cascode transistor is in the high-gain region.

**Fig. 4 — $f_T$ vs. gm/Id:** Use this to verify that the transistors are fast enough for the target GBW. As a rule of thumb, aim for $f_T > 10 \times$ GBW for each transistor in the signal path to keep parasitic poles well above unity gain. $f_T$ peaks in moderate inversion and drops rapidly toward weak inversion.

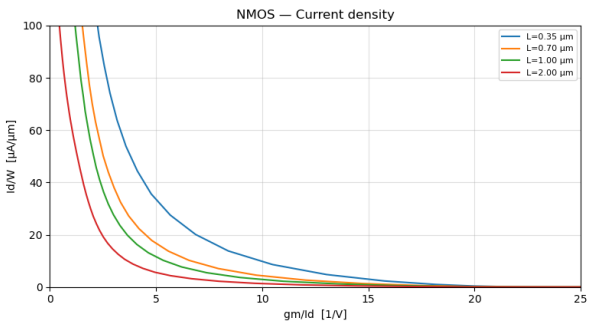


Fig. 1. Current density $I_D/W$ vs. $g_m/I_D$ for four NMOS channel lengths. Read the current density at the chosen $g_m/I_D$ target, then apply (4) to find width.

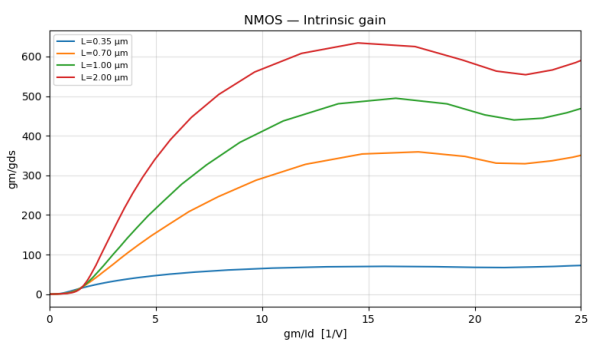


Fig. 2. Intrinsic gain $g_m/g_{ds}$ vs. $g_m/I_D$ for four NMOS channel lengths. Longer channels yield higher gain but reduce $f_T$.

## VIII. Conclusion

A Python-based gm/Id characterization and query tool has been presented. The tool automates the characterization sweep, stores results in a four-dimensional LUT, and provides a design-mode query that converts a $g_m/I_D$ target and required current into a transistor width in a single step. The methodology was demonstrated by sizing the cascode and output-stage transistors of a folded-cascode class-AB operational amplifier, including an explicit inversion-level trade-off for the output devices. Because all sizing data come directly from the SPICE model, the results are more accurate than square-law estimates, and minimal hand-calculation iteration is required before running the verification simulation.

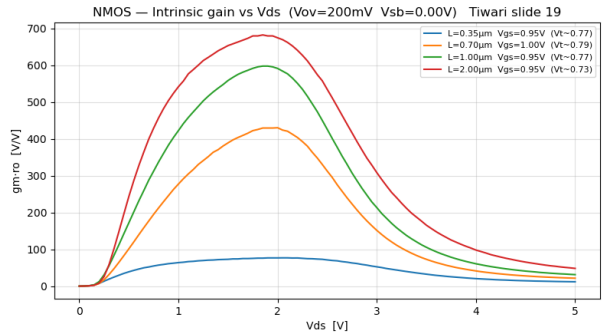


Fig. 3. Intrinsic gain $g_m r_o$ vs. $V_{DS}$ at $V_{ov} = 200\,$mV, $V_{SB} = 0$. Gain peaks near $V_{DS} \approx 2\,$V and falls at both extremes.

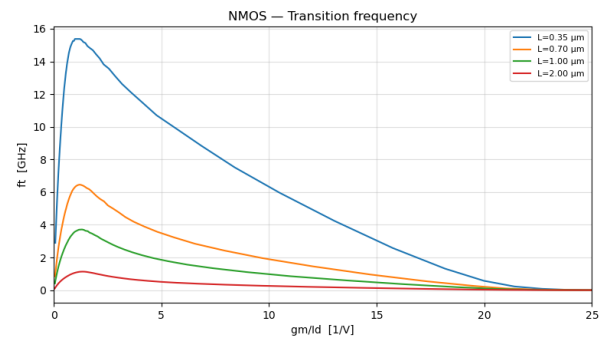


Fig. 4. Transition frequency $f_T$ vs. $g_m/I_D$. Higher $f_T$ requires stronger inversion (lower $g_m/I_D$) and draws more quiescent current. Minimum-length devices are fastest.


## Acknowledgment

The author thanks the Department of Electrical Engineering at Tarleton State University for providing access to simulation resources in addition to Dr. Eric Wyers for his help with this project.


## References


[1] F. Silveira, D. Flandre, and P. G. A. Jespers, "A gm/id based methodology for the design of CMOS analog circuits and its application to the synthesis of a silicon-on-insulator micropower OTA," *IEEE Journal of Solid-State Circuits*, vol. 31, no. 9, pp. 1314–1319, Sep. 1996.

[2] B. Murmann, "CMOS transistor sizing: Gm/Id approach," YouTube, Circuit Insights (ISSCC 2026), Episode 5, 2026, [Online]. Available: https://www.youtube.com/watch?v=oNDmJ2a3tWI.

[3] N. Hoki, "PyLTSpice: A Python interface for LTspice simulation," GitHub, 2023, [Online]. Available: https://github.com/nunobrum/PyLTSpice.